

La librería NumPy en Python

[NumPy](#) es una librería de Python especializada en el cálculo numérico y el análisis de datos, especialmente para un gran volumen de datos.

Incorpora una nueva clase de objetos llamados **arrays** que permite representar colecciones de datos de un mismo tipo en varias dimensiones, y funciones muy eficientes para su manipulación.

La ventaja de Numpy frente a las listas predefinidas en Python es que el procesamiento de los arrays se realiza mucho más rápido (hasta 50 veces más) que las listas, lo cual la hace ideal para el procesamiento de vectores y matrices de grandes dimensiones.



Previamente hay que instalar en el terminal del equipo (CMD, Powershell o Linux como administrador) los siguientes comandos:

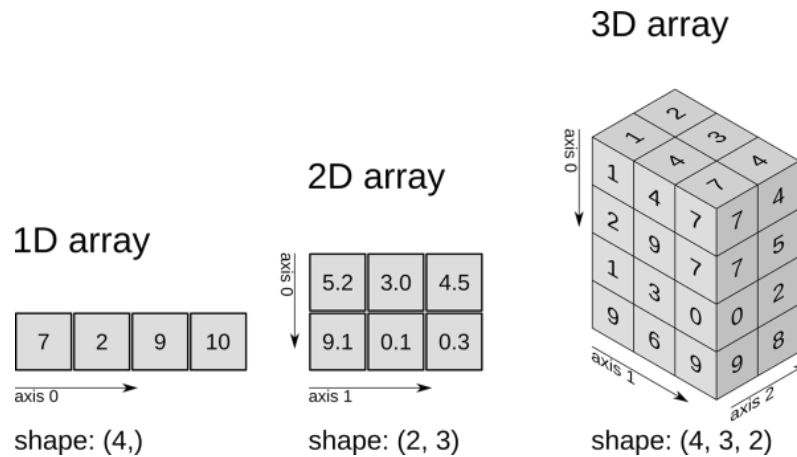
- `pip install numpy`

Para importar la librería Numpy y sus componentes debemos adjuntar al principio del programa, función o procedimiento que usemos la siguiente línea:

- `import numpy as np`

La clase de objetos array

Un array es una estructura de datos de un mismo tipo organizada en forma de tabla o cuadrícula de distintas dimensiones. Las dimensiones de un array también se conocen como **ejes**.



Creación de arrays

Para crear un array se utiliza la siguiente función de NumPy

- **np.array(lista):** Crea un array a partir de la lista o tupla lista y devuelve una referencia a él. El número de dimensiones del array dependerá de las listas o tuplas anidadas en lista:
- Para una lista de valores se crea un array de una dimensión, también conocido como **vector**.
- Para una lista de listas de valores se crea un array de dos dimensiones, también conocido como **matriz**.
- Para una lista de listas de listas de valores se crea un array de tres dimensiones, también conocido como **cubo**.
- Y así sucesivamente. No hay límite en el número de dimensiones del array más allá de la memoria disponible en el sistema.

Los elementos de la lista o tupla deben ser del mismo tipo.

```
>>> # Array de una dimensión
```

```
>>> a1 = np.array([1, 2, 3])
```

```
>>> print(a1)                                →      [1 2 3]
```

```
>>> # Array de dos dimensiones
```

```
>>> a2 = np.array([[1, 2, 3], [4, 5, 6]])
```

```
>>> print(a2)                                →      [[1 2 3]
                                                [4 5 6]]
```

```
>>> # Array de tres dimensiones
```

```
>>> a3 = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
```

```
>>> print(a3)                                →      [[[ 1  2  3]
                                                [ 4  5  6]]

                                                [[ 7  8  9]
                                                [10 11 12]]]
```

Otras funciones útiles que permiten generar arrays son:

- **np.empty(dimensiones):** Crea y devuelve una referencia a un array vacío con las dimensiones especificadas en la tupla dimensiones.
- **np.zeros(dimensiones):** Crea y devuelve una referencia a un array con las dimensiones especificadas en la tupla dimensiones cuyos elementos son todos ceros.
- **np.ones(dimensiones):** Crea y devuelve una referencia a un array con las dimensiones especificadas en la tupla dimensiones cuyos elementos son todos unos.

- **np.full(dimensiones, valor):** Crea y devuelve una referencia a un array con las dimensiones especificadas en la tupla dimensiones cuyos elementos son todos valor.
- **np.identity(n):** Crea y devuelve una referencia a la matriz identidad de dimensión n.
- **np.arange(inicio, fin, salto):** Crea y devuelve una referencia a un array de una dimensión cuyos elementos son la secuencia desde inicio hasta fin tomando valores cada salto.
- **np.linspace(inicio, fin, n):** Crea y devuelve una referencia a un array de una dimensión cuyos elementos son la secuencia de n valores equidistantes desde inicio hasta fin.
- **np.random.random(dimensiones):** Crea y devuelve una referencia a un array con las dimensiones especificadas en la tupla dimensiones cuyos elementos son aleatorios.

Ejemplos:

```
>>> print(np.zeros(3,2))      →      [[0. 0. 0.]
                                     [0. 0. 0.]]
```

```
>>> print(np.identity(3))     →      [[1. 0. 0.]
                                     [0. 1. 0.]
                                     [0. 0. 1.]]
```

```
>>> print(np.arange(1, 10, 2)) →      [1 3 5 7 9]
```

```
>>> print(np.linspace(0, 10, 5)) →      [ 0.  2.5  5.  7.5 10. ]
```

Atributos de un array

Existen varios atributos y funciones que describen las características de un array.

- **a.ndim:** Devuelve el número de dimensiones del array a.
- **a.shape:** Devuelve una tupla con las dimensiones del array a.
- **a.size:** Devuelve el número de elementos del array a.
- **a.dtype:** Devuelve el tipo de datos de los elementos del array a.

Acceso a los elementos de un array

Para acceder a los elementos contenidos en un array se usan índices al igual que para acceder a los elementos de una lista, pero indicando los índices de cada dimensión separados por comas. Al igual que para listas, los índices de cada dimensión comienzan en 0.

También es posible obtener subarrays con el operador dos puntos (:), indicando el índice inicial y el siguiente al final para cada dimensión, de nuevo separados por comas.

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]])
```

```
>>> print(a[1, 0]) →      4      # Acceso al elemento de la fila 1 columna 0
```

```
>>> print(a[1][0]) →      4      # Otra forma de acceder al mismo elemento
```

```
>>> print(a[:, 0:2]) →    [[1 2]
                          [4 5]]
```

Filtrado de elementos de un array

Una característica muy útil de los arrays es que es muy fácil obtener otro array con los elementos que cumplen una condición.

- `tabla[condicion]`: Devuelve una lista con los elementos del array a que cumplen la condición [condición]. Ejemplo

```
>>> tabla = np.array([[1, 2, 3], [4, 5, 6]])
>>> print(a[(tabla % 2 == 0)])           →      [2 4 6]

>>> print(a[(tabla % 2 == 0) & (tabla >= 4)]) →      [4 6]
```

Diferencias con los Dataframes

Los Dataframes obtenidos en Python por la lectura de ficheros (TXT, CSV, etc) o por recepción de datos de una base de datos (MySQL, SQL Server, etc) no se pueden modificar. Un dataframe estándar sería del tipo siguiente:

```
>>> antiguo = list([1, 3, 5, 7, 9, 2, 4, 6, 8])    # Dataframe, que no se puede modificar

>>> antiguo[0] = 20                                # Esto no se puede hacer, da error de ejecución!!!
```

Sin embargo si se puede modificar cuando realizamos la creación de un Array o Tupla usando NumPy:

```
>>> nuevo = np.array([1, 3, 5, 7, 9, 2, 4, 6, 8]) # Array que si se puede modificar
>>> nuevo[0] = 20
>>> print(nuevo)      → [20 3 5 7 9 2 4 6 8 ]
```

Ejemplos de uso de NumPy

- Para pasar una lista dataframe a un Array tipo NumPy

```
>>>nuevo = np.array(antiguo)
```

- Creación de una tabla con NumPy

```
>>>tabla = np.array([[1, 3, 5], [7, 9, 2], [4, 6, 8], [10, 12, 9]])
```

```
>>>print (tabla)      →      [[1 3 5]
                                [7 9 2]
                                [4 6 8]
                                [10 12 19]]
```

- Para obtener el número de columnas de una tabla

```
>>>columnas = len(tabla[0])      →      columnas = 3
```

- Para obtener el número de filas de una tabla

```
>>>filas = len(tabla)            →      filas = 4
```

- Para imprimir los valores de la tabla usando tabulaciones podemos usar el mismo sistema de recorrido que para un dataframe:

```
for i in range(filas):
    linea=""
    for j in range(columnas):
        linea = linea + str(tabla[i][j]) + "\t"
    print (linea + "\n")
```

Extracción y copia de datos en ficheros

- **Cargar datos desde un fichero**

```
>>>nuevo = np.loadtxt(carpetas + fichero, dtype=str, delimiter="\t")
```

Si no le indicamos el tipo de dato, por defecto toma tipo float, y si no le indicamos el delimitador, por defecto toma el espacio.

- **Descarte de filas**

Para descartar filas a importar usamos skiprows. Por ejemplo, podemos descartar las primeras 2 filas:

```
nuevo = np.loadtxt(carpetas + fichero, skiprows=2, dtype=str, delimiter="\t")
```

```
filas = len(nuevo)
```

```
columnas = len(nuevo[0])
```

```
for i in range(filas):
```

```
    linea=""
```

```
    for j in range(columnas):
```

```
        linea = linea + str(nuevo[i][j]) + "\t"
```

```
    print (linea + "\n")
```

- **Descarte de columnas**

Para importar columnas concretas usamos `usecols`. Por ejemplo, podemos importar solo las columnas de marca, modelo y matricula.

```
nuevo = np.loadtxt(carpeta + fichero, usecols=(0,1,4), dtype=str, delimiter="\t")
```

- **Para guardar una tabla de NumPy en un fichero de texto**

Hay que colocar tantos valores `%s` como columnas tengamos (pueden ser `%s` para cadenas, `%n` para números enteros, o `%d` para números decimales).

```
np.savetxt(carpeta + "prueba.csv", nuevo, delimiter=';', fmt=['%s','%s','%s'])
```

- **Para incorporar encabezados.**

Primero creamos una tabla nueva con el encabezado y luego las unimos usando **CONCATENATE**. Es imprescindible que las tablas tengan la misma cantidad de columnas.

```
cabecera = np.array(["Marca", "Modelo", "Año", "Color", "Matricula"])
completo = np.concatenate((cabecera, nuevo), axis=0)
np.savetxt(carpeta + "concabecera.csv", completo,
           delimiter=';', fmt=['%s','%s','%s','%s','%s'])
```

Operaciones básicas con NumPy

- **Para imprimir los valores de la tabla usando tabulaciones**

```
filas = len(nuevo)
columnas = len(nuevo[0])

for i in range(filas):
    linea=""
    for j in range(columnas):
        linea = linea + str(nuevo[i][j]) + "\t"
    print (linea + "\n")
```

- **Usando métodos especiales de los Arrays de NumPy**

- **Número de elementos del array (filas * columnas)**

```
print("El array tiene " + str(nuevo.size) + " elementos")
```

- **Número de dimensiones del array**

```
print("Tiene " + str(nuevo.ndim) + " dimensiones")
```

- **Para crear una copia del array de NumPy y obtener filas y columnas**

```
copia = np.copy(nuevo)
print("Sus parámetros son " + str(copia.shape))
print("Es decir, tiene " + str(copia.shape[0]) + " filas") # Número de filas
print("Y tiene " + str(copia.shape[1]) + " columnas") # Número de columnas
```

- **Para modificar un array de NumPy**

```
copia[0,0]="HOLA"
copia[0,1]="ESTO"
copia[0,2]="ES"
copia[0,3]="UNA"
copia[0,4]="COPIA"
```

- **Imprimiendo el array copia en el terminal usando métodos de NumPy**

```
for i in range(copia.shape[0]):
    linea=""
    for j in range(copia.shape[1]):
        linea = linea + str(copia[i][j]) + "\t"
    print (linea + "\n")
```

Operaciones matemáticas con arrays

Existen dos formas de realizar operaciones matemáticas con arrays: a nivel de elemento y a nivel de array. Las operaciones a nivel de elemento operan los elementos que ocupan la misma posición en dos arrays. Se necesitan, por tanto, dos arrays con las mismas dimensiones y el resultado es un array de la misma dimensión.

Los operadores matemáticos básicos (+, -, *, /, %, **) se utilizan para la realizar suma, resta, producto, cociente, resto y potencia a nivel de elemento.

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]])
>>> b = np.array([[1, 1, 1], [2, 2, 2]])
>>> print(a + b )           →      [[2 3 4]
                                   [6 7 8]]
```

```
>>> print(a / b)      →      [[1.  2.  3. ]
                               [2.  2.5  3. ]]
```

```
>>> print(a ** 2)     →      [[ 1  4  9]
                               [16 25 36]]
```

Operaciones básicas en tuplas de 2 dimensiones

Siendo **nombre** el nombre de la tabla de NumPy podemos usar los siguientes métodos:

```
>>> nombre = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
>>> recorre = nombre.shape
>>> filas = len(nombre)
>>> columnas = len(nombre[0])
```

También podemos saber la cantidad de filas y columnas así:

```
>>> filas = recorre[0]
>>> columnas = recorre[1]
>>> print(nombre)    →      [[1 2 3]
                               [4 5 6]
                               [7 8 9]]
```

Cuando realicemos cálculos con funciones, y el resultado vaya a ser decimal hay que indicarlo con el parámetro **dtype=np.float64**. Si no lo hacemos el resultado quedará redondeado a decimal.

- **Media de todos los valores de la matriz**

```
media = np.mean(nombre, dtype=np.float64)
```

- **Media de la columna 3**

```
mediacol = np.mean(nombre[:, 2], dtype=np.float64)
```

- **Media de la fila 1**

```
mediafil = np.mean(nombre[0, :], dtype=np.float64)
```

- **Para calcular medias parciales en filas o columnas**

- **Media de la columna 2, solo de las filas 2 y 3**

```
media_parcial_col = mediacol = np.mean(nombre[1:3, 2], dtype=np.float64)
```

- **Media de la fila 1, solo de las columnas 2 y 3**

```
media_parcial_fil = np.mean(nombre[0, 1:3], dtype=np.float64)
```

- **Desviación estándar y varianza**

```
desv_estandar = np.std(nombre, dtype=np.float64)
```

```
varianza = np.var(nombre, dtype=np.float64)
```

```
desv_std_col = np.std(nombre[:, 2], dtype=np.float64) # Desviación estándar columna
```

```
varianza_fila = np.var(nombre[0, :], dtype=np.float64) # Varianza por fila
```

- **Mínimos y máximos**

```
minimo = nombre.min()
```

```
maximo = nombre.max()
```

```
minimo_col = nombre[:, 2].min() # Mínimo por columna
```

```
maximo_fila = nombre[0, :].max() # Máximo por fila
```

- **Suma y producto de los elementos, suma y producto acumulativos**

```
suma = np.sum(nombre, dtype=np.float64)
```

```
producto = np.prod(nombre, dtype=np.float64)
```

- **Suma acumulativa. Recorre por filas.**

```
suma_fila = np.cumsum(nombre, dtype=np.float64)
```

- **Producto acumulativo. Recorre por filas.**

```
prod_fila = np.cumprod(nombre, dtype=np.float64)
```

Álgebra matricial

Numpy incorpora funciones para realizar las principales operaciones algebraicas con vectores y matrices. La mayoría de los métodos algebraicos se agrupan en el submódulo linalg. Por tanto antes de usarla debemos importarla en el programa:

```
>>>import numpy as np
>>>import numpy.linalg as la
```

Operaciones con matrices

- **Producto escalar de dos vectores**

Para realizar el producto escalar de dos vectores **se utiliza el operador @** o el método **DOT**. Ejemplo, **u.dot(v)** Devuelve el producto escalar de los vectores u y v:

```
>>> a = np.array([1, 2, 3])
>>> b = np.array([1, 0, 1])
>>> print(a @ b)      →      4
>>> print(a.dot(b))   →      4
```

- **Módulo de un vector**

Para calcular el módulo de un vector se utiliza el método **NORM**. Ejemplo, **norm(v)** devuelve el módulo del vector v.

```
>>> a = np.array([3, 4])
>>> print(np.linalg.norm(a)) →      5.0
```

- **Producto de dos matrices**

Para realizar el producto matricial se utiliza **el mismo operador @ o el método DOT** que se usa para el producto escalar de vectores. Ejemplo, **a.dot(b)** devuelve el producto matricial de las matrices a y b siempre y cuando sus dimensiones sean compatibles.

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]])
>>> b = np.array([[1, 1], [2, 2], [3, 3]])
>>> print(a @ b)      →      [[14 14]
                               [32 32]]
>>> print(a.dot(b))   →      [[14 14]
                               [32 32]]
```

- **Traza de una matriz**

La traza de una matriz cuadrada se calcula con el método **TRACE**. Ejemplo, **a.trace()** Devuelve la traza (suma de la diagonal principal) de la matriz cuadrada a.

```
>>> import numpy as np
>>> a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
>>> print(a.trace())   →      15
```

- **Determinante de una matriz**

El determinante de una matriz cuadrada se calcula con la función **DET**. Ejemplo, **det(a)** devuelve el determinante de la matriz cuadrada a.

```
>>> a = np.array([[1, 2], [3, 4]])
>>> print(np.linalg.det(a)) →      -2.0
```

- **Matriz traspuesta**

Para transponer una matriz se utiliza **el método T**. Ejemplo, **a.T** devuelve la matriz traspuesta de la matriz a.

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]])
>>> print(a.T)           →      [[1 4]
                                   [2 5]
                                   [3 6]]
```

- **Matriz inversa**

La inversa de una matriz se calcula con la función **INV**. Ejemplo, **inv(a)** devuelve la matriz inversa de la matriz cuadrada a.

```
>>> a = np.array([[1, 2], [3, 4]])
>>> print(np.linalg.inv(a))      →      [[-2.  1.]
                                           [ 1.5 -0.5]]
```

- **Autovalores de una matriz**

Los autovalores de una matriz cuadrada se calculan con la función **EIGVALS**. Ejemplo, **eigvals(a)** devuelve los autovalores de la matriz cuadrada a.

```
>>> a = np.array([[1, 1, 0], [1, 2, 1], [0, 1, 1]])
>>> print(np.linalg.eigvals(a))
→      [ 3.00000000e+00  1.00000000e+00 -3.36770206e-17]
```

- **Autovectores de una matriz**

Los autovectores de una matriz cuadrada se calculan con la función **EIG**. Ejemplo, **eig(a)** devuelve los autovalores y los autovectores asociados de la matriz cuadrada a.

```
>>> a = np.array([[1, 1, 0], [1, 2, 1], [0, 1, 1]])
>>> print(np.linalg.eig(a))      →
(array([ 3.00000000e+00,  1.00000000e+00, -3.36770206e-17]),
 array([[-4.08248290e-01,  7.07106781e-01,  5.77350269e-01],
        [-8.16496581e-01,  2.61239546e-16, -5.77350269e-01],
        [-4.08248290e-01, -7.07106781e-01,  5.77350269e-01]]))
```

- **Aplicación de matrices a la solución de un sistema de ecuaciones**

Para resolver un sistema de ecuaciones lineales se utiliza la función **SOLVE**. Ejemplo, **solve(a, b)** devuelve la solución del sistema de ecuaciones lineales con los coeficientes de la matriz a y los términos independientes de la matriz b.

Por ejemplo, tenemos un sistema de dos ecuaciones y dos incógnitas con las siguientes expresiones:

$$\begin{aligned}x + 2y &= 1 \\ 3x + 5y &= 2\end{aligned}$$

Podemos resolverlo usando matrices, creando una matriz a para los índices X e Y, y otra b para los resultados. A ambas les aplicamos la función SOLVE:

```
>>> a = np.array([[1, 2], [3, 5]])
>>> b = np.array([1, 2])
>>> print(np.linalg.solve(a, b)) →      [-1.  1.]
```